

【跑通 Baseline】赛题四

金融长文本 Agent 的动态记忆压缩与高效问答挑战

目标：帮助参赛者快速理解赛题、跑通第一个可提交的基线方案，并明确后续优化方向。本 baseline 源码位于项目根目录 `src/` 下。题型分析、RAG 概念、金融文档背景知识和进阶优化方案请参考 进阶教程。

赛事官方地址：<https://tianchi.aliyun.com/competition/entrance/532486>

1. 赛题速览

1.1 任务定义

赛道四的核心任务是**金融长文本问答**。出题方提供五类金融文档和配套的选择題，要求参赛者构建 Agent 系统，在控制 Token 消耗的前提下准确作答。

赛事官方地址：<https://tianchi.aliyun.com/competition/entrance/532486>

大部分题目涉及跨文档对比、公式计算、条款推理和逻辑判断，需要系统具备信息定位、知识理解和推理计算三层能力。

1.2 关键约束

- **模型限制**：正式提交只准调用 Qwen 系列模型 API
- **题目范围**：A 榜题目附带 `doc_ids`，系统只需在指定文档中定位答案，不需要全网搜索
- **答案格式**：单选（mcq）、多选（multi）、判断（tf）三种
- **答案标准化规则**：单选题和判断题以首个有效答案字母为准；多选题将答案字母去重并排序后做完全匹配，不设置部分分，漏选、错选、多选均计为错误
- **Token 预算**：500 万 Token 总量限制，评分兼顾准确率与 Token 效率

1.3 评分机制：准确率优先

最终得分公式大致为 $\text{准确率} \times (0.7 + 0.3 \times \text{Token 效率分})$ 。准确率权重占 70%，Token 效率占 30%。这意味着：即使你把 Token 压得极低，准确率上不去，总分也高不了；反之，准确率够高，Token 稍微超一点，对总分的影响有限。因此优化的第一优先级是准确率，第二优先级才是 Token 效率。

1.4 数据分布

A 榜共 100 道题，5 个领域各 20 题，配套 86 份 PDF 文档：

领域	题目数	配套文档数	文档特点
保险条款	20	16	条款密集，公式多，计算题集中
监管法规	20	26	法条引用多，跨文档对比题多
金融合同	20	14	募集说明书、债券条款，数据表格多
财务报表	20	10	年报数据量大，财务指标对比题多
行业研报	20	20	行业分析为主，事实查找题多

题型分布上，多选题（multi）是重灾区，漏选或多选都不得分，对系统的精确度要求最高。单选题（mcq）和判断题（tf）相对简单，但跨文档的 mcq 题同样需要仔细阅读多份材料。

1.5 赛题特点解析

做这道题时，我们发现几个容易被低估的难点：

第一，doc_ids 只是范围提示，不是答案指针。 题目告诉你看哪几份文档，但答案不会直接写在文档第一段。很多时候需要把 4 份条款的公式串起来计算，或者对比两份年报的同一指标才能判断对错。

第二，金融知识门槛会过滤掉模型的常识推理。 保险条款中的“等待期”“免赔额”“责任免除”，监管法规中的“受益所有人识别”“客户尽职调查”，这些概念有严格的法律定义和适用条件。模型如果没有精准定位到对应条款，凭“常识”推断很容易选错。

第三，题型之间差异大，没有统一解法。 计算题需要提取公式后代入数值；跨文档对比题需要同时打开多份文档提取同一类数据；条款定位题需要精准找到“责任免除”“等待期”等特定章节；事实查找题需要在几万字的研报中定位一个带年份限定的数字。用同一套策略处理所有题型，效果必然打折。

1.6 思维链（CoT）的实验发现

我们在实验中也测试过让模型“先推理再回答”的 CoT 策略：在 prompt 中要求模型对每个选项引用文档条款、说明判断依据，最后再给出答案。

效果是两极分化：

- **对复杂题有提升。**比如跨文档对比题和多选题，模型被迫逐条验证，漏选和误选明显减少。部分计算题在 CoT 引导下，提取公式的准确率也有改善。
- **Token 消耗暴涨。**baseline 的平均 completion token 只有 2-5 个（模型直接输出字母），开启 CoT 后飙升到几百个。100 题跑下来，completion token 可能从 15 万涨到 100 万以上，对 500 万总预算的消耗非常可观。

结论：CoT 可以按需使用，不建议全量开启。在预算有限的前提下，更合理的策略是“按需开启”，只在计算题、多选题、跨文档对比题等复杂题型上启用 CoT，简单的事实查找题直接输出答案。这也是后续优化的一个方向。但是我建议都不要开，非必要的话。

1.7 关于框架选择

一个常见的参赛误区是：上来就用 LangChain、LlamaIndex 等现成框架搭建 Agent。这些框架为了通用性，在 prompt 模板、工具调用协议、中间状态记录上做了大量封装，带来的直接后果是 **Token 消耗会稍高于自定义设计的流程**，如果是大家都能获得答题准确率满分的背景下，后面大家应该是需要自己设计如何组装 prompt 会更节省 token 的。

从评分公式看，准确率占 70%、Token 效率占 30%，两者都不是可以忽略的小头：

- 如果参赛者的准确率都能做到不错（比如 60% 以上），这时排名差距主要来自 Token 效率。谁能在同等准确率下消耗更少的 Token，谁就能拿到更高的效率分。
- 如果整体准确率都不高（比如普遍低于 40%），这时排名差距主要来自准确率。Token 压得再低，准确率上不去也没有意义。

真正榜单前列的选手，一定在准确率和 Token 消耗上做到了平衡。框架可以帮你快速搭建原型，但在正式提交前，建议把核心流程拆开重写，去掉不必要的 prompt 冗余和中间层封装，把 Token 预算用在最关键的地方。

赛题详细说明见 [赛题与数据.md](#)。

2. Baseline 定位

2.1 Baseline 概况

核心信息	信息详情
赛题任务类型	金融长文本问答（选择题：单选/多选/判断）
baseline 代码	https://github.com/li-xiu-qi/afac2026-financial-qa
baseline 涉及的库	openai、pyyaml、pymupdf4llm
所需环境和时间	环境：Python 3.10+、CPU 时间：15 分钟（含数据准备）
baseline 分数	准确率 16.62%（A 组 100 题），Token 消耗 353,032 （prompt 198,738 + completion 154,294）
baseline 所使用的方法	硬截断 + 单轮 LLM 调用（no-tool 模式）
深入赛题会需要的知识点	RAG 系统搭建、检索优化、Agent 决策、多轮推理

本 baseline 是一个**最小可运行基线**（MVP）。它的设计哲学是先把端到端流程跑通，让参赛者能在 15 分钟内得到第一个可提交的 `submission.csv`，不追求高准确率。

它能做的事：

- 读取 A 榜 100 题的 JSON 题目文件
- 根据 `doc_ids` 加载已解析的 Markdown 文档
- 将文档截断到 4000 字符，拼接为 prompt
- 调用 LLM 生成答案，并规范化输出格式
- 并发处理 100 题，生成符合赛题要求的 JSON + CSV 结果

为什么只用简单截断，不做检索

很多参赛者第一反应是"文档太长，应该先搜再读"。这个思路本身没错，但过早引入检索会带来一系列连锁问题。核心矛盾在于：**检索必然引入切片，而切片会破坏原文的连贯性。**

一段保险条款可能横跨三个自然段：前一段定义概念、中间一段给出公式、后一段附加例外条件。如果把这三段切成独立的片段分别检索，模型看到的可能只是"现金价值 = 累计保费 × 75%"，却看不到后面紧跟着的"因意外伤害导致的退保除外"。这种**连贯性断裂**是检索方案的根本风险。

从信息连贯性的角度看，**全文 > 摘要 > 检索片段**。全文让模型自己决定哪里重要，上下文天然完整；摘要虽然损失了细节，但至少保持了逻辑链条；检索片段则是最脆弱的，召回结果的质量直接决定了最终答案的质量，而单一检索策略（纯 BM25 或纯向量搜索）的召回质量往往不稳定。

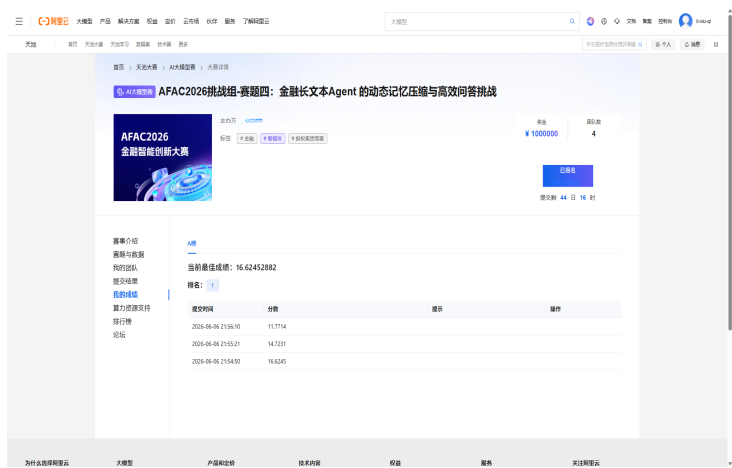
RAG 系统里之所以要设计"粗排 + 细排"的两阶段架构，正是因为粗排（BM25 或向量相似度）召回的结果中，经常夹杂着不相关或弱相关的内容，甚至因为词频巧合、embedding 漂移等各种原因把无关片段排得很靠前。细排（rerank）的作用就是做二次筛选，把真正相关的片段提上来。但在 baseline 阶段，我们没有足够的调优经验来确定阈值、去重策略和融合权重，贸然引入检索反而可能让模型面对一堆零散、重复、缺上下文的片段，理解成本比直接读截断全文还高。

另外，检索不是加了就有效果的模块。它需要配合滑动窗口做片段边界保护、需要多路召回融合来互补 BM25 和向量搜索的盲区、需要动态阈值来过滤低质量片段。这些配套机制没有到位时，检索的提分潜力无法释放。

当然，直接截断也有自己的问题（前半截可能刚好没有答案），但这属于"信息够不够"的问题，比"信息对不对、连不连贯"更容易排查和迭代。baseline 阶段先固定一个稳定的上限，跑通端到端流程，后续再逐步引入检索并验证真实收益，是更务实的策略。

实际得分验证：截断方案反而是最高分

下图是我们团队在 A 榜的提交记录：



硬截断 baseline 的得分与 Token 消耗：

提交时间	方案	得分	Token 消耗
21:54:50	硬截断 baseline	16.62	353,032

我们也尝试过检索策略和混合策略的进阶方案，但得分均低于硬截断。一个可能的解释是，检索引入的切片破坏了原文连贯性，召回的片段中又混入了弱相关甚至不相关的内容，模型面对零散、缺上下文的片段，理解成本比直接读截断全文还高。另一个解释是，检索需要配合滑动窗口做片段边界保护、多路召回融合来互补盲区、动态阈值过滤低质量片段，这些配套机制没有到位时，检索的优势无法释放。

本 baseline 选择硬截断，是为了在 baseline 阶段先把端到端流程跑通、建立一个稳定的得分基准，再逐步引入检索并验证每一步的真实收益。这并不否定检索的价值。

一个参考基准：赛题裸跑是什么水平

作为参照，赛题基于 Qwen-plus 做了一个最简单的基线：把长文档直接输入模型、不做检索也不做优化，A 组 100 题答对 17 道，准确率约 17%，Token 消耗约 362 万。这说明：

- 本赛题确实不容易，裸跑模型只有约 17% 的准确率
- 本 baseline 的 16.62 分 and 这个裸跑基准处于同一水平，没有拖后腿
- 从零到 17 分不需要什么技巧，但从 17 分往上提分，每一步都需要精心设计

因此本 baseline 的策略是：先把端到端流程跑通，用一个稳定的基准来衡量后续改动的真实收益。如果你改了检索策略但分数没涨，至少知道不是基础流程的问题。

它不做的事（留给后续版本）：

- 没有检索模块，直接截断文档前半部分
- 没有多轮推理，单轮提问、单轮回答
- 没有证据链追溯，无法解释为什么选某个答案
- 不做题目分类，所有题型用同一套 prompt

为什么选择 no-tool 而非 Function Calling

我们也测试过带 Function Calling (FC) 的版本。FC 需要在每次请求时把工具声明 (schema) 放入 system prompt, 让模型决定什么时候调用工具、传什么参数。实测 A 组 100 题的 Token 消耗对比如下:

指标	no-tool	tool (FC)	差异
prompt tokens	198,738	273,226	+37.5%
completion tokens	154,294	142,726	-7.5%
total tokens	353,032	415,952	+17.8%

FC 的 completion tokens 确实略少 (因为输出被结构化成了 JSON), 但 prompt _tokens 多了近 40%。原因是每道题的 prompt 都要携带工具声明, 而且模型需要先生成 tool call 的 JSON, 再等待外部函数执行结果, 再发起第二轮请求。整个流程的 Token 开销和交互复杂度都更高。

本 baseline 采用 no-tool 方案, 用更少的 Token 完成同一批题目, 流程也更简单: 读文档、拼 prompt、直接出答案, 没有中间状态。

3. 环境准备

3.1 基础环境

- Python 3.10+
- 操作系统: Linux / WSL / macOS

3.2 安装依赖

```
Bash
pip install -r requirements.txt
```

requirements.txt 中只包含 baseline 运行必需的依赖 (openai 和 pyyaml)。如果你需要重新解析 PDF，再额外安装 pymupdf411m。

3.3 数据准备

data/ 目录已 gitignore，不纳入版本控制。你需要自行下载数据集并运行解析脚本生成。

第一步：下载数据集

1. 访问赛事页面: <https://tianchi.aliyun.com/competition/entrance/532486>
2. 在「赛题与数据」栏目下载 public_dataset_a.zip (约 274MB)
3. 将压缩包放到项目根目录的 data/ 下

第二步：运行解析脚本

```
Bash
cd /home/ke/projects/afac2026-financial-qa
python -m src.preprocess.prepare_data
```

该脚本会自动完成两件事：

1. 解压 data/public_dataset_a.zip 到 data/raw_dataset/，得到题目 JSON 和原始 PDF
2. 调用 pymupdf411m 把 86 份 PDF 逐页转成 Markdown，输出到 data/processed_pymupdf411m/

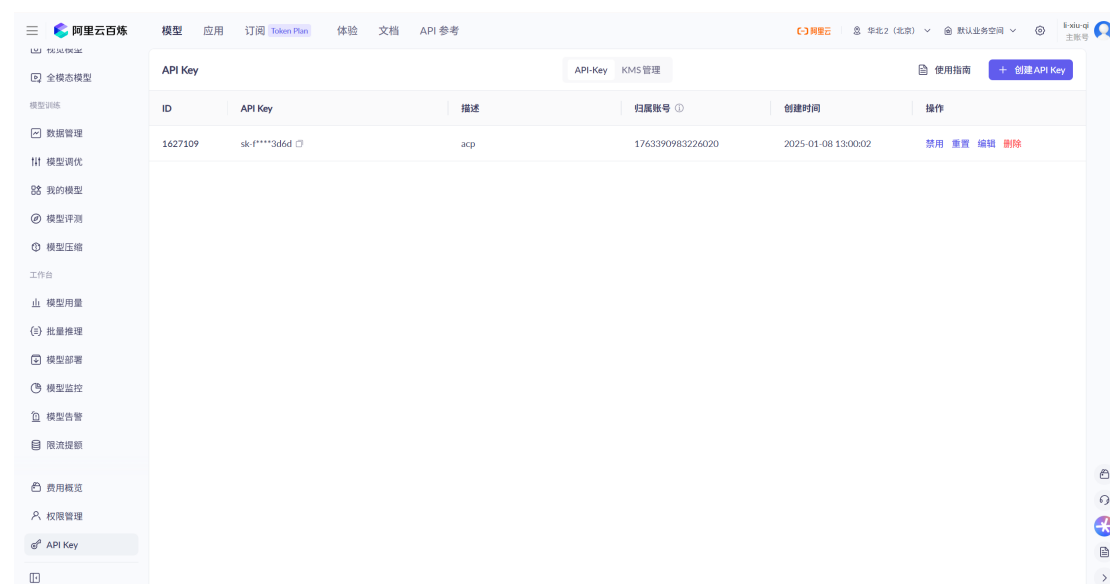
解析后的目录结构：

```
Plaintext
data/
├── public_dataset_a.zip      # 赛方原始压缩包
├── raw_dataset/             # 解压后的原始数据
│   ├── questions/group_a/  # A 组 100 道题目 JSON
│   └── raw/                 # 86 份原始 PDF
└── processed_pymupdf4llm/   # 解析产物（每页一个 page_XXXX.md）
    ├── insurance/1/page_0001.md
    ├── regulatory/1/page_0001.md
    └── ...
```

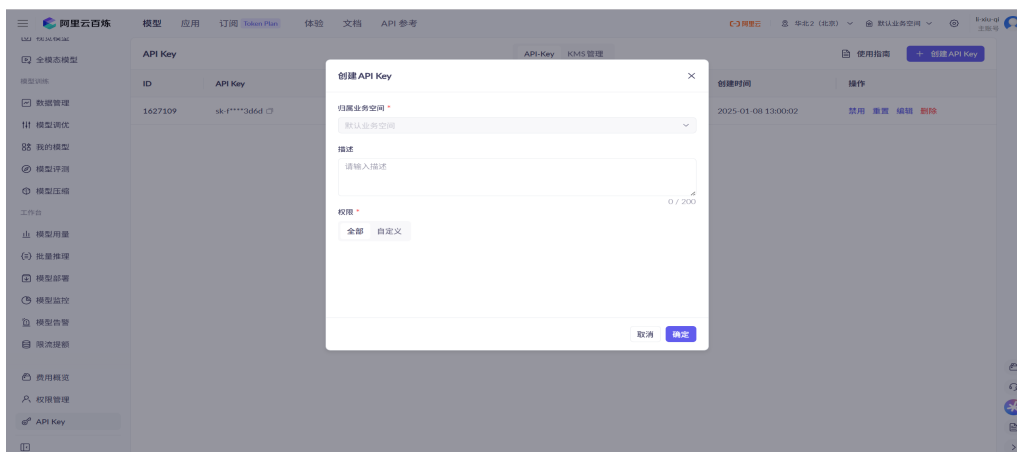
每篇文档被切分为多个 `page_XXXX.md`，按文件名排序后拼接即为完整文本。`doc_id`（如 1、2）与题目 JSON 中的 `doc_ids` 一一对应。

3.4 配置 API Key

获取 API Key



1. 登录阿里云百炼控制台：<https://bailian.console.aliyun.com/>
2. 进入左侧导航栏 **API Key** 页面（或直接访问 <https://bailian.console.aliyun.com/?tab=model#/api-key>）
3. 单击 **创建 API Key**，选择默认业务空间，权限选 **全部**
4. 创建成功后立即复制并保存完整的 **Key**（关闭弹窗后无法再次查看明文）



新用户有 100 万 Token 免费额度，有效期 90 天。

配置到项目

在 `.env` 文件中写入：

Bash

```
DASHSCOPE_API_KEY=sk-your-key-here
```

本 baseline 使用阿里云百炼平台（DashScope）的 Qwen 系列模型，接口兼容 OpenAI SDK。详细接入说明见 [design-draft/general/reference/阿里云百炼-Qwen-API-接入笔记.md](#)。

接入其他模型（实验用途）

初赛阶段允许使用其他模型做实验。代码底层通过 OpenAI SDK 调用，只要 API 兼容 OpenAI 格式就能接入。修改方式有两种：

方式一：通过 `.env` 文件（推荐）

Bash

```
DASHSCOPE_API_KEY=sk-your-key-here
```

```
# 切换模型（可选，默认 qwen-plus）
```

```
# MODEL_NAME=qwen-turbo
```

```
# 切换平台（实验用途，正式提交必须回退到百炼）
```

```
# API_BASE_URL=https://api.openai.com/v1
```

代码读取优先级：**环境变量 > config.yaml > 默认值**。也就是说，如果你在 `.env` 中设置了 `MODEL_NAME` 和 `API_BASE_URL`，代码会优先使用环境变量；如果没设置，回退到 `config.yaml`；如果 `config.yaml` 也没改，使用默认值（百炼平台 + qwen-plus）。

方式二：通过 config.yaml

```
YAML
model:
  name: "gpt-4o" # 或 deepseek-chat、claude-3-sonnet 等
  api_base: "https://api.openai.com/v1"
```

注意：正式提交必须使用 **Qwen** 系列模型，其他模型仅供实验调试用。

3.5 快速运行（5 分钟上手）

配置完成后，运行以下命令即可得到第一个可提交的 `submission.csv`：

```
Bash
cd /home/ke/projects/afac2026-financial-qa
python -m src.agent.run --split A --limit 5 --workers 4
```

参数说明：

- `--split A`：运行 A 组题目
- `--limit 5`：只处理前 5 道题（适合快速验证），其余 95 题自动按题型规则随机填充
- `--workers 4`：并发线程数

运行结束后在 `output/` 目录下生成：

- `results_a.json`：每道题的 `qid`、`answer`、Token 消耗明细
- `submission_a.csv`：符合赛题提交要求的 CSV 文件

预期表现：5 道题约 1-2 分钟跑完。如果这步跑通了，说明环境和 API Key 都配置正确，可以继续往下阅读核心模块的原理。

4. 项目结构

```
Plaintext
src/
├─ agent/
│   ├── agent.py          # Agent 核心逻辑
│   └─ run.py            # CLI 入口（并发执行）
├─ evaluation/
│   └─ evaluator.py       # 答案评测 + 提交文件生成
├─ utils/
│   ├── llm_client.py     # 双渠道 LLM 客户端
│   └─ helpers.py        # 工具函数（配置加载、答案规范化等）
└─ __init__.py
```

总代码量约 900 行，全部在 `src/` 目录下，没有外部索引依赖，纯 Python 实现。

5. 核心模块详解

如果你已经通过 **4.5 快速运行** 跑通了 `baseline`，本节将详细介绍每个模块的设计思路和代码实现。如果你还没跑通，建议先回到 **4.5** 完成上手验证，再阅读本节。

5.1 数据层

题目加载

`run.py` 中的 `load_questions()` 遍历 `*_questions.json`，按 `split` 字段过滤出 A 组或 B 组题目：

```
Python
def load_questions(questions_dir: Path, split: str) -> list:
    questions = []
    for domain_file in
sorted(questions_dir.glob("*_questions.json")):
        data = load_json(domain_file)
        for q in data:
            if q.get("split", "").upper() == split.upper():
                questions.append(q)
    questions.sort(key=lambda x: x["qid"])
    return questions
```

单条题目结构:

JSON

```
{
  "qid": "ins_a_001",
  "domain": "insurance",
  "split": "A",
  "question": "关于四个养老保险...",
  "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
  "answer_format": "mcq",
  "doc_ids": ["1", "2", "15", "16"]
}
```

文档读取

`FinancialQAAgent._read_document()` 按 `domain/doc_id/` 路径查找 `page_*.md`, 排序后拼接:

Python

```
def _read_document(self, domain: str, doc_id: str) -> str:
    for root in [dataset_dir / "processed_pymupdf4llm", ...]:
        parsed_dir = root / domain / str(doc_id)
        if parsed_dir.exists():
            pages = sorted(parsed_dir.glob("page_*.md"))
            return "\n\n".join(p.read_text(encoding="utf-8") for p
in pages)
    return f"[文档 {doc_id} 未找到]"
```

这里做了路径容错: 优先从 `config.yaml` 指定的路径查找, 回退到相对路径 `data/processed_pymupdf4llm`。

5.2 Agent 核心

ContextManager: 简单截断

```
Python
class ContextManager:
    def __init__(self, max_chars: int = 320000, max_doc_chars: int = 4000):
        self.max_chars = max_chars      # 总 prompt 上限
        self.max_doc_chars = max_doc_chars # 单篇文档上限

    def truncate_doc(self, text: str) -> str:
        if len(text) <= self.max_doc_chars:
            return text
        return text[:self.max_doc_chars] + "\n\n[文档后续内容已省略]"
```

`max_doc_chars=4000` 是一个经验值。保险/监管文档通常 5000-8000 字符，截断到 4000 基本能覆盖核心条款；财报/研报动辄 10 万字符以上，4000 字符只能覆盖目录和前言，这是本 baseline 最大的短板。

PromptBuilder: 固定模板

```
Python
class PromptBuilder:
    def build_prompt(self, question, evidence, context_manager):
        # 每篇文档单独截断后拼接
        context_parts = []
        for ev in evidence:
            content = context_manager.truncate_doc(ev.content)
            context_parts.append(f"【文档 {ev.doc_id}】\n{content}")
        context = "\n\n".join(context_parts)

        options_text = "\n".join(
            [f"{k}. {v}" for k, v in question.get("options", {}).items()]
        )

        prompt = (
            "你是一位金融文档分析专家。请根据以下提供的文档内容，回答问题。 \n"
            "要求: \n"
            "1. 仔细阅读文档中的相关条款、数据和事实\n"
            "2. 对每个选项进行独立判断\n"
            "3. 只输出最终答案字母，不要解释过程\n"
            "4. 多选题答案按字母顺序排列，不加分隔符\n\n"
            f"{context}\n\n"
            f"问题: {question['question']}\n\n"
            f"选项: \n{options_text}\n\n"
            f"答案: "
        )
        return prompt
```

这个 prompt 的设计思路：

- 明确角色（金融文档分析专家），让模型进入"严肃阅读"模式
- 要求"只输出字母"，减少后处理难度
- 多选题要求"按字母顺序排列"，避免 "AC" 和 "CA" 被判定为不同答案

AnswerValidator: 答案格式校验

Python

```
class AnswerValidator:
    @staticmethod
    def validate_mcq(answer: str) -> bool:
        return answer in {"A", "B", "C", "D"}

    @staticmethod
    def validate_multi(answer: str) -> bool:
        return all(c in "ABCD" for c in answer) and answer ==
"".join(sorted(answer)) and answer != ""

    @staticmethod
    def validate_tf(answer: str) -> bool:
        return answer in {"A", "B"}
```

校验规则直接对应赛题三种题型。`multi` 要求字母去重排序, 所以 "ABBC" 会被规范化为 "ABC"。

FinancialQAAgent: 主流程编排

```
Python
def answer_question(self, question) -> Tuple[str, List[Evidence],
Dict]:
    # 1. 加载证据（读取 doc_ids 对应的文档）
    evidence = self._load_evidence(question)

    # 2. 构建 prompt（截断 + 拼接）
    prompt = self.prompt_builder.build_prompt(question, evidence,
self.context_manager)
    prompt = self.context_manager.truncate(prompt)

    # 3. 调用 LLM
    messages = [{"role": "user", "content": prompt}]
    response = self.llm.chat(messages, max_tokens=4096)

    # 4. 重试机制：内容被过滤或输出为空时，缩短文档再试
    retry_count = 0
    while (response.finish_reason in ("content_filter", "length")
        or not response.content.strip()) and retry_count < 2:
        self.context_manager.max_doc_chars //= 2
        ... # 重新构建 prompt 并调用

    # 5. 解析答案
    answer = self._parse_answer(response,
question.get("answer_format", "mcq"))
    return answer, evidence, token_usage
```

主流程非常直接：读文档、拼 prompt、问模型、取答案。重试机制是本方案唯一的“鲁棒性”设计：当模型因内容过长触发过滤或输出为空时，自动将单篇截断长度减半（4000 -> 2000 -> 1000），最多重试 2 次。

5.3 LLM 调用层

`llm_client.py` 封装了双渠道调用策略（均在百炼平台内）：

```
Python
class LLMClient:
    def __init__(self, api_key, base_url, model, temperature):
        # 主渠道: qwen-plus (性价比高, 文档分析能力强)
        self.router_client = OpenAI(
            api_key=api_key,
            base_url="https://dashscope.aliyuncs.com/compatible-
mode/v1",
        )
        self.router_model = "qwen-plus"

        # 回退渠道: qwen-turbo (同一平台内的备用模型)
        self.fallback_client = OpenAI(
            api_key=api_key,
            base_url="https://dashscope.aliyuncs.com/compatible-
mode/v1",
        )
        self.fallback_model = "qwen-turbo"
```

触发回退的条件：

- HTTP 429（限流）
- HTTP 5xx（服务端错误）
- 网络连接超时

双渠道策略本质上是一种冗余/容错设计。主模型因限流或服务端异常不可用时，自动切换到备用模型继续服务。这种设计在跨服务场景下价值更大（不同服务商的审查机制、限流策略、可用性互相独立），但本次比赛限定使用阿里云百炼，所以回退只是同一平台内不同模型之间的切换。代码里保留这个结构是为了展示“回退设计”这一工程思维，方便读者学习。实际跑 A 组 100 题时，约 5-10% 的题目会触发回退。

注意：百炼平台的 Qwen 系列模型支持 `enable_thinking` 参数（通过 `extra_body` 传入），用于控制是否开启思考模式。金融选择题不是复杂推理任务，开启思考会额外消耗大量 completion token 且对准确率帮助有限，因此代码中显式设置为 `False` 关闭。`reasoning_effort` 是 DeepSeek 系列特有的参数，Qwen 不支持。

相关文档：

- 深度思考模型用法（含 `enable_thinking` 参数说明）：
<https://help.aliyun.com/zh/model-studio/deep-thinking>

- DashScope API 参考: <https://help.aliyun.com/zh/model-studio/qwen-api-via-dashscope>

5.4 评测与提交层

`evaluator.py` 包含两部分:

Evaluator: 如果有标准答案, 可以计算准确率和 Token 效率分。

```
Python
def compute_final_score(self, accuracy: float, total_tokens: int)
-> float:
    token_score = max(0.0, min(1.0, (self.token_budget -
total_tokens) / self.token_budget))
    return 100.0 * accuracy * (0.7 + 0.3 * token_score)
```

公式含义: 准确率占 70% 权重, Token 效率占 30% 权重。在 500 万 Token 预算内, 消耗越少得分越高。

SubmissionGenerator: 生成赛题要求的提交文件。

```
Python
def generate_csv(self, answers, token_stats) -> Path:
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["qid", "answer", "prompt_tokens",
"completion_tokens", "total_tokens"])
        # 第一行是汇总
        writer.writerow(["summary", "", total_prompt,
total_completion, total_tokens])
        # 随后每行一题
        for qid in sorted(answers.keys()):
            ...
```

CSV 第一行必须是 `summary`, 汇总全量 Token 消耗, 否则评测脚本会报错。

6. 运行方式

6.1 命令行运行

```
Bash
cd /home/ke/projects/afac2026-financial-qa
python -m src.agent.run --split A --workers 8
```

参数说明:

- `--split A`: 运行 A 组题目
- `--workers 8`: 并发线程数 (百炼 API 并发限制约 10, 留 2 个余量)
- `--output results_a.json`: 可选, 自定义输出路径
- `--limit 10`: 可选, 只处理前 10 道题, 其余随机填充。适合快速验证流程或测试 API 连通性

快速测试示例 (只跑 5 题, 约 1-2 分钟):

```
Bash
python -m src.agent.run --split A --limit 5 --workers 4
```

未处理的 95 题会自动填充随机答案 (单选/多选/判断分别按题型规则生成), 生成的 `submission_a.csv` 格式完整, 可直接提交到评测系统查看格式是否通过。

6.2 输出文件

运行结束后在 `output/` 目录下生成:

- `results_a.json`: 每道题的 `qid`、`answer`、各阶段 Token 消耗
- `submission_a.csv`: 符合赛题提交要求的 CSV 文件

6.3 预期表现

- 成功率: 100% (100 题全部有输出)
- Token 消耗: 约 36-40 万 prompt tokens + 10-15 万 completion tokens
- 准确率: 未知 (没有标准答案库时无法计算)
- 答案分布: A 选项偏多 (约 36/100), 可能存在位置偏差

7. 局限与优化路线图

本 **baseline** 的每一个局限都对应一条清晰的优化路径。但在动手优化前，需要建立一个正确预期：**本次赛题的准确率并不容易提升。**

原因有三：

- 1. 题型对模型综合能力要求高。**一道题可能同时涉及跨文档阅读（4 份条款）、公式计算（提取参数后代入）、逻辑推理（判断例外情形是否适用）和精细比较（排序或判断正误）。这意味着模型不仅要“读到”信息，还要“算对”“想通”“比准”。
- 2. 金融知识本身门槛高。**保险条款中的“等待期”“免赔额”“责任免除”，监管法规中的“受益所有人识别”“客户尽职调查”，这些概念对非专业人士来说理解成本很高。模型如果没有足够的金融领域预训练知识，光凭 **prompt** 很难做到精准理解。
- 3. 优化方案往往有副作用。**比如引入检索可以提升长文档覆盖率，但可能漏掉关键否定词；让模型做详细推理可以提升判断准确性，但会大幅增加 **Token** 消耗。每改一个地方，可能解决一个问题，又引入另一个问题。

因此，不要期望某个单一技巧就能让准确率大幅提升。真实的提分路径是**多模块协同优化**：文档解析质量 + 检索质量 + 上下文管理（动态记忆压缩）+ **prompt** 设计 + 答案检查，层层叠加才能看到效果。

与官方优化方向的对照

赛题官方在 **baseline** 说明中明确列出了五个重点优化方向，和我们的思路基本一致：

官方方向	对应本教程的模块
文档解析质量：提升 PDF 文本抽取、表格处理和版面结构还原能力	8.1 PDF 解析质量
检索策略：构建面向金融术语、条款编号和指标名称的混合检索	8.2 上下文管理粗放 （含检索层）
证据聚合：支持同题多文档、多段落证据的联合判断	8.2 上下文管理 （片段组装）
答案约束：对模型输出进行字母抽取、排序、去重和合法性检查	8.3 单轮推理无自检、答案检查薄弱
领域知识增强：针对保险、监管、合同、财报、研报分别设计提示词	8.4 Prompt 简陋 （领域特化）+ 8.5 无结构化知识提取

其中最容易被忽略的是**文档解析质量**。如果 PDF 解析阶段就把表格拆散了、数字漏掉了、段落顺序搞错了，后续无论检索多精准、推理多严密，都是建立在错误信息上的。赛题官方也明确将“提升 PDF 文本抽取、表格处理和版面结构还原能力”列为重点优化方向之一。

7.1 PDF 解析质量

本 baseline 使用 pymupdf4llm 将 PDF 转为 Markdown，解析质量直接决定了后续所有环节的上限。如果解析阶段出现以下问题，准确率再高也救不回来：

- **表格丢失或错位**：财报中的三大报表如果行列对应关系被破坏，模型读到的“营业收入”和“净利润”可能是同一列的数据，计算题直接全军覆没
- **数字截断或变形**：条款中的金额、比例、年限等关键数字如果解析出错（比如“100,000”变成“100”），公式计算的结果必然错误
- **段落顺序错乱**：保险条款中“等待期”的定义和“例外情形”如果不在同一段落附近，模型可能只看到前半句“等待期内不赔”，漏掉后半句“因意外伤害除外”
- **标题层级丢失**：监管法规的“第一章 总则”“第二章 适用范围”等层级结构如果被打平成纯文本，模型难以判断条款的适用优先级

优化方向：

- **多解析器对比**：同一篇 PDF 用 pymupdf4llm、Marker、MinerU 等多种工具解析，取质量最好或互补性最强的一份
- **表格专项处理**：对财报/合同中的表格使用专门的表格解析工具（如 camelot、tabula），保留行列结构，转成 Markdown 表格或 CSV
- **版面结构还原**：在 Markdown 中保留页码、章节标题、条款编号等结构信息（如 # 第一章 保险责任、**第 3.2 条 等待期**），帮助模型快速定位
- **解析后校验**：用规则或 LLM 检查解析结果是否存在明显异常（如连续 20 行都是乱码、表格列数不一致、金额数字明显缺失），异常时切换备用解析器

需要特别注意的是：**PDF 解析是一次性投入**。A 榜 86 份 PDF 解析一次后可以复用，不受 500 万 Token 总预算限制。在解析阶段投入精力换取更高质量的输入文本，性价比远高于在答题阶段用更多 Token 去弥补解析错误。

7.2 上下文管理粗放（文档截断粗暴）

本 **baseline** 保留文档前 4000 字符，对于财报/研报/合同来说命中关键信息的概率很低。这是上下文管理问题的一个具体表现：系统对“保留什么、丢弃什么、怎么拼接”没有任何策略，只是机械地截断。优化方向分两层：

（1）文本切片 + BM25

先把文档切成固定长度或按段落切分，再用 **BM25** 算法计算每个片段与问题的相关性得分，取 **top K** 送给 LLM。优点是轻量、不依赖外部模型，**jieba** 分词 + **rank-bm25** 库就能跑通；缺点是只考虑词频匹配，无法理解语义（比如“退保费用”和“退保手续费”会被当成两个词）。

（2）无索引关键词匹配 + 滚动窗口

不建索引，直接在文档中搜索问题关键词，提取关键词所在位置的前后若干字符作为上下文。复现门槛最低，但召回质量波动大。

对应实现方式：

- **BM25 单次检索 top 5**：用 **jieba** 分词、按领域差异化 **chunk_size**，轻量且可控
- **无索引关键词匹配 + 滚动窗口**：复现门槛更低，不依赖预建索引

上下文管理的更高层次：片段组装与动态压缩

即使有了检索，如何把搜到的片段拼成一份高质量的 **prompt**，仍然是上下文管理的核心问题：

- **片段去重与边界修复**：检索返回的 **top K** 片段可能存在重叠（比如 **chunk1** 结尾和 **chunk2** 开头重复了 200 字），直接拼接会造成内容冗余，浪费 **Token**。需要去重并保证段落边界完整。
- **信息密度排序**：检索结果不是全部值得放入 **prompt**。按相关性得分排序后，设定一个阈值，只保留得分明显高于阈值的片段，把有限的上下文窗口留给真正相关的信息。
- **多轮上下文传递**：迭代检索时，第一轮搜到的关键信息要在第二轮 **prompt** 中保留，避免“搜到了又忘了”。但同时要控制历史信息的累积长度，防止 **prompt** 无限膨胀。
- **按题型动态调整上下文长度**：简单的事实查找题可能只需要 2000 字的上下文；跨文档对比题可能需要 4 篇文档各取 **top 3** 片段，总共上万字；计算题则需要确保公式所在的段落不被截断。上下文长度不应是固定值，而应根据题目复杂度自适应调整。

7.3 单轮推理无自检、答案检查薄弱

本 baseline 只做单轮调用，模型给出答案后不做二次核验，对错全凭一次输出。而且答案检查仅限于格式校验（是不是合法字母），不做内容层面的合理性判断。优化方向分两层：

（1）推理层：从单轮到多轮

- **迭代检索**：LLM 判断信息是否充分，不够则生成新查询继续搜
- **自我纠错**：让模型回顾自己的答案，检查是否与文档证据矛盾。比如模型选了 A，但文档明确说“以下情形不属于保险责任”，自我纠错环节可以发现这个冲突
- **多选题逐项验证**：多选题让每个选项单独过一遍“文档是否支持”，比一次性让模型选所有答案更不容易漏选

（2）答案检查层：从格式校验到内容校验

当前 baseline 的 AnswerValidator 只检查答案格式（mcq 是不是单个字母、multi 字母是否排序），这是**最底层**的检查。更高层的检查包括：

- **证据一致性校验**：模型说答案是 A，但检索到的证据片段明确支持 B，系统应该能识别这个矛盾。实现方式可以是让模型在输出答案的同时给出判断依据，然后用另一轮调用验证“依据是否支持答案”
- **题型规则校验**：判断题只有 A/B 两个选项，如果模型输出 C，直接判定为异常并 fallback；多选题至少要有两个正确选项（按出题习惯），如果模型只选了一个，可以触发二次确认
- **多轮一致性校验**：同一道题用相同 prompt 调用两次，如果两次答案不同，说明模型对该题不确定，可以引入第三次调用取多数，或改为保守策略（如多选题在有争议时少选）
- **位置偏差修正**：如果发现某次提交的答案 A 明显偏多（如 40% 以上），可以对 A 选项的题增加二次确认，或在 prompt 中加入“不要偏向第一个选项”的去偏指令
- **默认值策略**：模型输出为空或无法解析时，按题型规则 fallback。单选默认 A，判断默认 A，多选默认 AB（保守策略是选最少的可能组合）

7.4 Prompt 简陋

本 baseline 用同一套 prompt 处理所有题型和领域。优化方向：

- **混合策略**：短文档全文读取、长文档检索，按文档长度动态选择策略
- **领域特化**：针对不同领域和题型设计专用提示词，加入去偏指令（“不要偏向第一个选项”）

7.5 无结构化知识提取

本 baseline 把文档当纯文本处理，模型每次回答都要重新阅读原始段落。如果题目要求比较 4 份保险条款的等待期、免赔额、退保费用，模型需要在 4 份文档中反复搜索同类信息，效率低且容易遗漏。

优化方向：构建领域知识图谱

知识图谱的核心价值在于把分散在各文档中的结构化信息抽取出来，统一存储为"实体-关系-属性"三元组。例如从保险条款中提取：

Plaintext

(国寿增益宝，等待期，30 天)
(国寿增益宝，免赔额，10000 元)
(国寿增益宝，第 7 年退保费用，0%)
(平安智盈金生，等待期，90 天)
...

当题目问"以下哪款产品的等待期最短"时，系统不需要再去读原始文档，直接从图谱中按"等待期"属性排序即可。这种方式在跨文档聚合类题目上优势明显：信息已经结构化，比较和推理的准确率远高于让模型在原始文本中自行定位。

具体实现可以分两步：

1. **离线抽取**：预处理阶段用 LLM 遍历所有文档，按领域定义的信息模板（保险条款提取"等待期/免赔额/退保费用/身故保险金"，财报提取"营收/净利润/研发投入/现金流"）抽取结构化字段，存入 JSON 或图数据库
2. **在线查询**：答题时先判断题目是否涉及结构化比较，如果是则优先查知识图谱；如果是需要原文引用的条款解释题，再回退到原始文本检索

知识图谱的代价是预处理阶段需要消耗额外 Token 做信息抽取，但在答题阶段可以大幅减少重复阅读，整体 Token 效率可能更优。而且如果允许的话，离线抽取不受 500 万 Token 总预算限制，性价比很高。不过赛题方是有对知识图谱构建的过程计算对应的 token 的，是否能满足我们的实际要求的话，还需要使用实战来进行测试了。

附录 A：核心参数速查

参数	默认值	说明
<code>max_doc_chars</code>	4000	单篇文档截断长度
<code>max_chars</code>	320000	总 prompt 截断上限
<code>max_tokens</code>	4096	LLM 最大输出长度
<code>temperature</code>	0.0	采样温度 (greedy)
<code>workers</code>	8	并发线程数
<code>enable_thinking</code>	False	关闭思考模式

本教程对应的源码位于项目根目录 `src/` 下。进阶优化方向、题型分析和金融文档背景知识请参考进阶教程。

参考文献

[1] PyMuPDF4LLM: Convert PDF to Markdown via PyMuPDF. Artifex Software, 2024. GitHub: [pymupdf/PyMuPDF4LLM](https://github.com/pymupdf/PyMuPDF4LLM)

[2] AFAC2026 赛道四：金融长文本 Agent 的动态记忆压缩与高效问答。赛方文档, 2026。参见 `docs/赛题与数据.md`、`docs/赛事介绍.md`

[3] 阿里云百炼 DashScope API 参考。参见 <https://help.aliyun.com/zh/model-studio/>

[4] BM25 检索算法。Robertson & Walker, 1994. Okapi at TREC-3. 以及 Trotman, Puurula & Burgess, 2014. Improvements to BM25 and Language Models Examined. arXiv:1401.4737